Análise de Eficácia de Testes com Teste de Mutação

Pontifícia Universidade Católica de Minas Gerais — Engenharia de Software

Disciplina: Teste de Software
Aluno: Vinicius Gomes Rodrigues
Repositório: https://github.com/viniciusgomesrod/operacoes-mutante

1. Configuração e execução

O projeto analisado é a biblioteca operacoes-mutante, com 50 funções matemáticas em src/operacoes.js e uma suíte Jest de 50 testes (test/operacoes.test.js), um por função.

- **Ambiente:** o npm install do StrykerJS 10 exibiu avisos EBADENGINE, pois essa versão exige Node ≥ 22 e a máquina usava o Node 20.13.1. Instalei o Node 22 com o nvm, reinstalei as dependências e os avisos desapareceram.
- **Instalação:** npm install --save-dev @stryker-mutator/core @stryker-mutator/jest-runner.
- **Configuração:** npx stryker init, escolhendo *None/other* (projeto sem framework), jest como test runner e os reporters html e clear-text.
- **Execução:** npx stryker run. O Stryker instrumentou 1 arquivo-fonte com **213 mutantes** e usou análise de cobertura perTest.

2. Análise inicial

Métrica	Valor
Cobertura de linhas (Jest)	98,64%
Cobertura de funções (Jest)	100%
Cobertura de statements (Jest)	85,41%
Cobertura de branches (Jest)	58,82%
Mutation score (Stryker)	73,71%

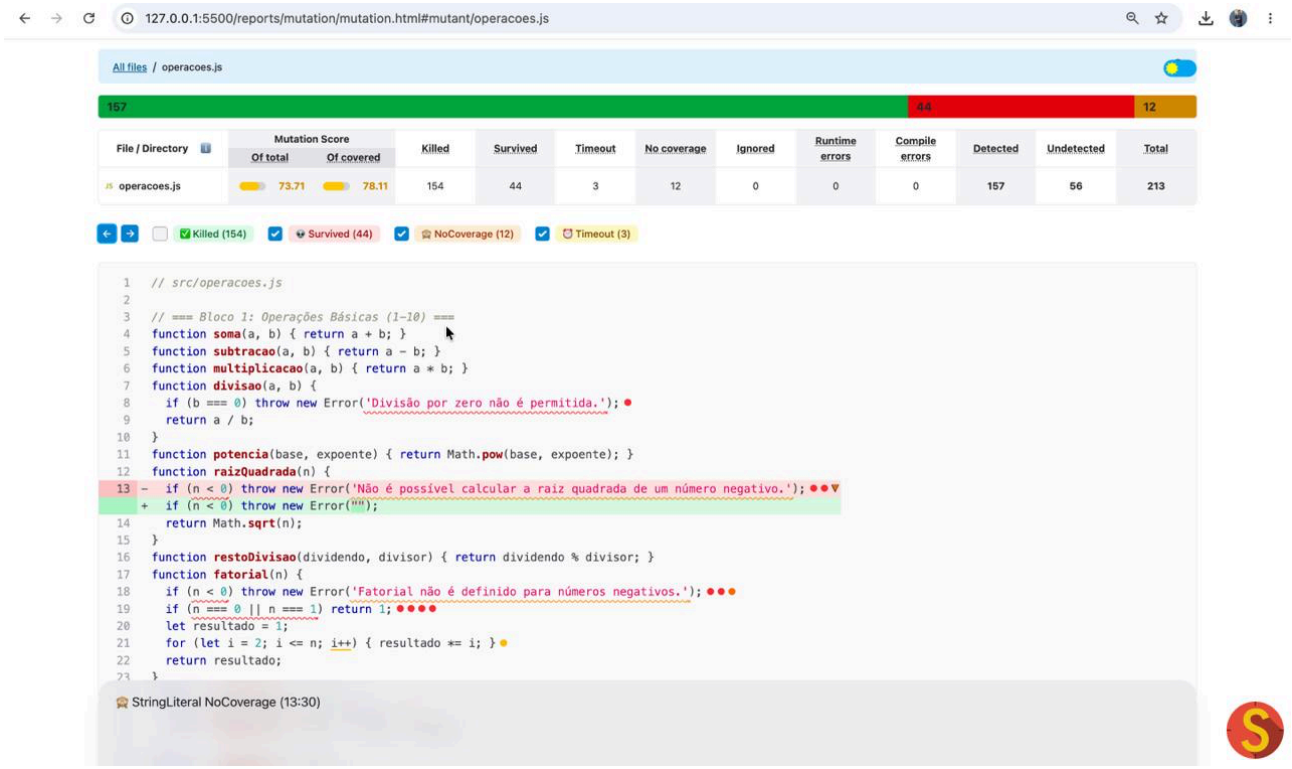
Resultado do Stryker na primeira execução: 154 mutantes mortos, 3 timeouts, **44 sobreviventes** e **12 sem cobertura** (NoCoverage), de um total de 213.

Discrepância. A cobertura de linhas, de 98,64%, sugere uma suíte quase completa, mas apenas 73,71% dos mutantes foram detectados, uma diferença de cerca de 25 pontos percentuais. O motivo é que a cobertura mede se o código foi **executado**, e não se alguém **verificou** o resultado. Os 50 testes trazem uma única asserção cada, quase sempre com entrada válida e resultado "feliz".

Dois detalhes ilustram isso:

- Em if (n < 0) throw new Error(...), o Jest considera a linha coberta, porque o if é avaliado. O Stryker, porém, marcou a mensagem do throw como **NoCoverage**, já que nenhum teste chega a executá-la.
- A cobertura de branches (58,82%) já era menor que o mutation score. Isso mostra que a métrica de linhas escondia o problema, enquanto a de branches e a análise de mutação o revelavam.

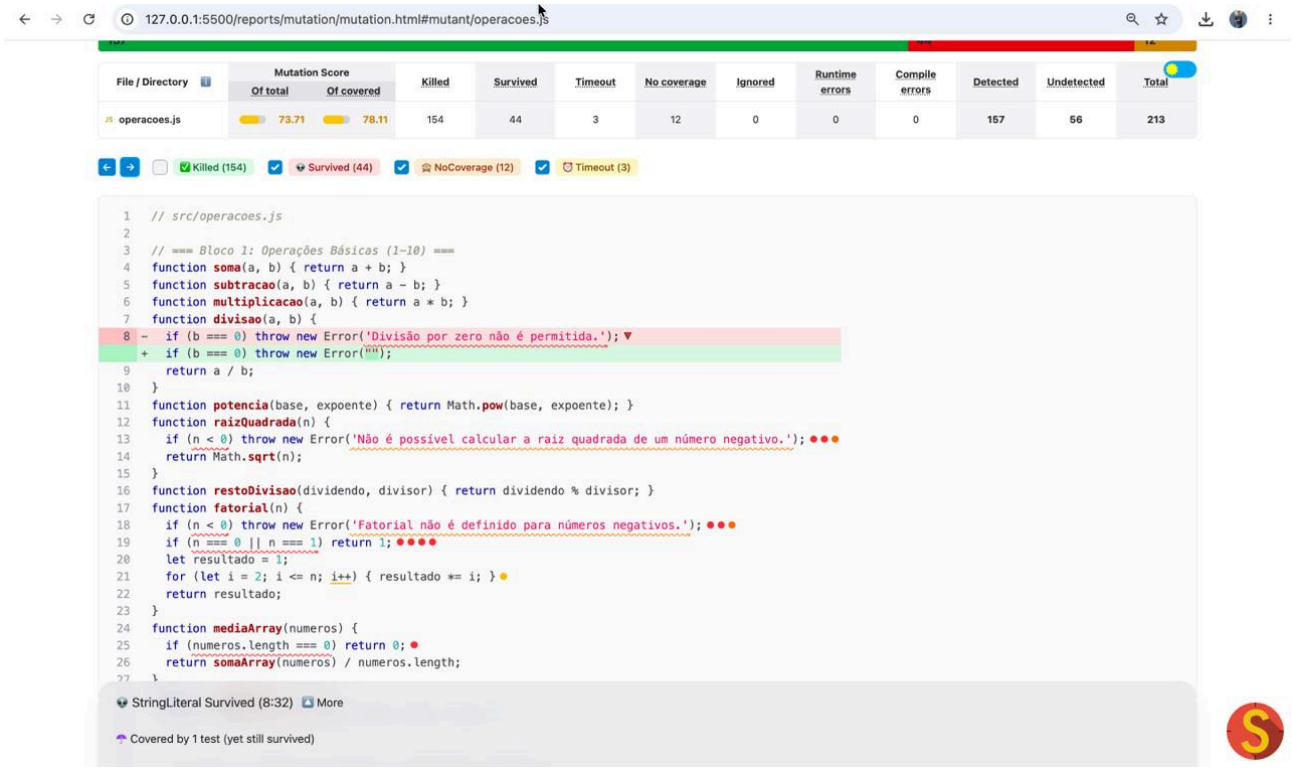
Mutante NoCoverage na linha 13 (StringLiteral)



3. Análise de mutantes críticos

Mutante 1 — Linha 8, StringLiteral (sobreviveu)

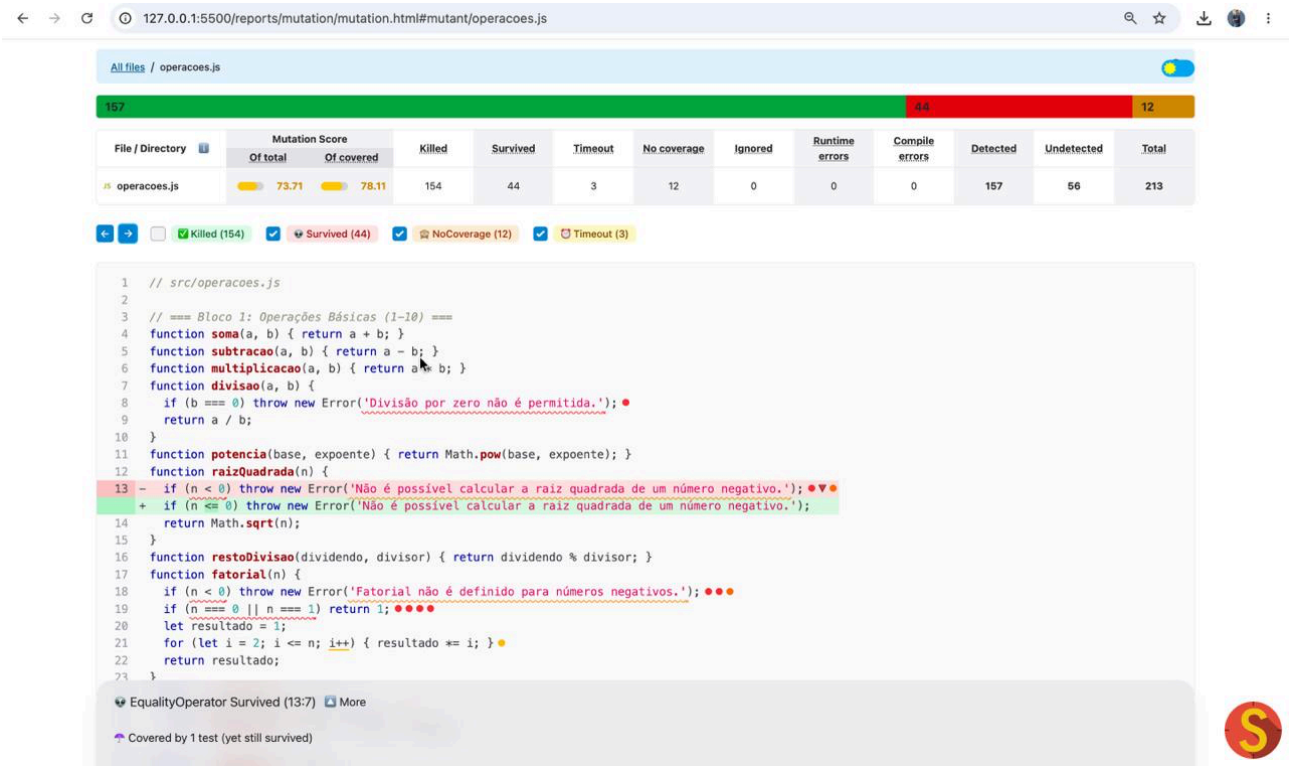
- if (b === 0) throw new Error('Divisão por zero não é permitida.');
+ if (b === 0) throw new Error("");



O que a mutação fez: trocou a mensagem de erro por uma string vazia. **Por que o teste original não matou:** o teste 4 usa expect(() => divisao(5, 0)).toThrow() sem argumento. Ele só verifica que **alguma** exceção é lançada, e não qual. A mudança da mensagem não altera esse resultado.

Mutante 2 — Linha 13, EqualityOperator (sobreviveu)

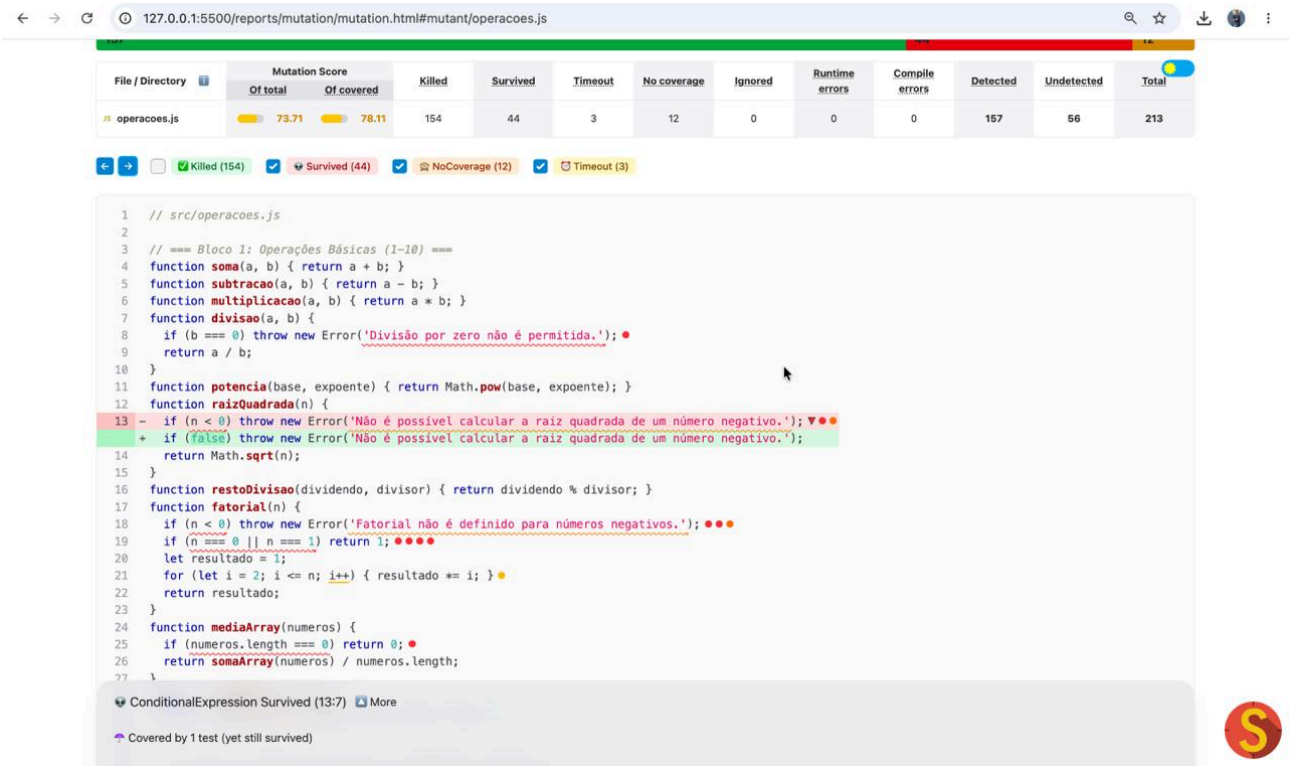
```
- if (n < 0) throw new Error('Não é possível calcular a raiz quadrada ...');
+ if (n <= 0) throw new Error('Não é possível calcular a raiz quadrada ...');
```



O que a mutação fez: trocou < por <=, passando a rejeitar o zero. **Por que o teste original não matou:** o único teste da função é raizQuadrada(16). O valor de fronteira (n = 0) nunca foi testado, então os dois comportamentos coincidem para todas as entradas exercitadas. É o caso clássico de < contra <= sem teste na fronteira.

Mutante 3 — Linha 13, ConditionalExpression (sobreviveu)

```
- if (n < 0) throw new Error('Não é possível calcular a raiz quadrada ...');
+ if (false) throw new Error('Não é possível calcular a raiz quadrada ...');
```



O que a mutação fez: substituiu a condição por false, removendo a validação por completo. **Por que o teste original não matou:** o relatório indica *"Covered by 1 test (yet still survived)"*: a linha foi executada, mas apenas com entrada válida, e o ramo de erro nunca foi tomado. A função ficou sem validação e nenhum teste percebeu. É o exemplo mais direto de **cobertura sem eficácia**.

4. Solução implementada

Criei o arquivo test/operacoes.mutantes.test.js, mantendo os 50 testes originais intactos. Os testes novos seguem o raciocínio "qual entrada faria o código mutado se comportar diferente do original?". Para os mutantes analisados:

```
// Mutante 1 (L8): a mensagem passa a ser verificada
expect(() => divisao(5, 0)).toThrow('Divisão por zero não é permitida.');

// Mutante 3 (L13): a entrada inválida precisa lançar a exceção
expect(() => raizQuadrada(-1))
  .toThrow('Não é possível calcular a raiz quadrada de um número negativo.');

// Mutante 2 (L13): o valor de fronteira é válido
expect(raizQuadrada(0)).toBe(0);
```

Por que são eficazes:

- O primeiro falha com a mensagem "", pois toThrow('texto') compara o conteúdo.
- O segundo falha com if (false), já que nenhuma exceção é lançada, e também com a mensagem vazia.
- O terceiro falha com n <= 0: o código original retorna 0 e o mutante lança um erro.

O mesmo raciocínio foi aplicado às demais lacunas do relatório inicial:

Tipo de lacuna	Testes adicionados
Entradas inválidas e mensagens de erro	fatorial(-1), inverso(0), maximoArray([]), minimoArray([]), medianaArray([])

Valores de fronteira	fatorial(0), isMaiorQue(5, 5), isMenorQue(5, 5), clamp(-5, 0, 10), clamp(15, 0, 10)
Apenas o caso positivo era testado	isPar(3), isImpar(4), isDivisivel(10, 3), isEqual(1, 2), isMaiorQue(5, 10), isMenorQue(10, 5)
Entrada que anula um termo da fórmula	celsiusParaFahrenheit(100), fahrenheitParaCelsius(212) (os testes originais usavam 0 e 32)
Entradas fáceis demais	medianaArray([3, 1, 2]) (desordenado, exige o sort), [1, 2, 3, 4] (ramo par)
Só números primos testados	isPrimo com -7, 0, 1, 2, 4, 9, 15
Array vazio sem teste	mediaArray([])

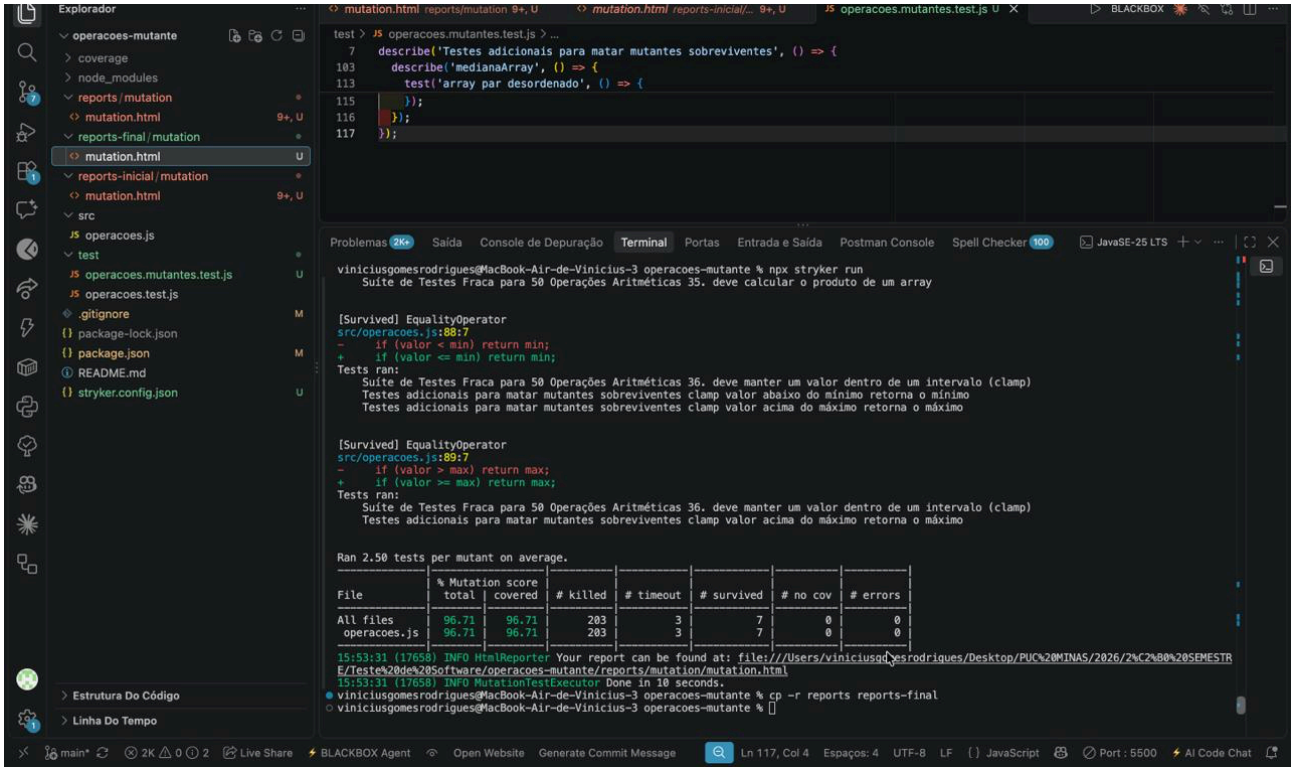
A suíte passou de 50 para 83 testes, e todos passam com o código original.

5. Resultados finais

	Inicial	**Final**
Testes	50	83
Mutantes mortos	154	203
Timeouts (contam como mortos)	3	3
Sobreviventes	44	7
NoCoverage	12	0
Mutation score (Stryker)	73,71%	96,71%

A melhoria foi de **+23 pontos percentuais**, com 49 mutantes a mais detectados e nenhum sem cobertura.

Resultado final do Stryker



Os 7 mutantes restantes são equivalentes

Os 7 sobreviventes não podem ser mortos por nenhum teste, porque o código mutado se comporta exatamente como o original:

Linha	Mutante	Por que é equivalente
19 (×4)	if (false), &&, false, ..., …, false	Com resultado = 1 e o laço começando em i = 2, n = 0 e n = 1 já retornam 1 sem o if. O if é redundante
84	if (false) return 1	reduce(..., 1) já retorna 1 para um array vazio
88	valor <= min	Com valor === min, retornar min ou valor dá o mesmo resultado
89	valor >= max	Idem para max

O Stryker não identifica mutantes equivalentes automaticamente, por isso eles ficam no cálculo do número bruto. Pela definição vista em aula, MS = 100 × D / (N − E):

*MS = 100 × 206 / (213 − 7) = **100%***

Considerando o ajuste mais conservador, que trata como equivalentes apenas os 5 das linhas 19 e 84 (nas linhas 88 e 89 a única diferença possível seria -0 contra 0), o resultado é 206 / 208 = **99,04%**, ainda acima da meta de 98%.

Observação sobre a meta. A meta de mais de 98% **não é alcançável pelo número bruto do Stryker (96,71%)**, nem com qualquer teste adicional, pelo motivo acima. Ela é atingida quando se aplica a fórmula da disciplina, que desconta os equivalentes. Os dois valores estão apresentados neste relatório por transparência.

6. Conclusão

O trabalho mostrou, na prática, que cobertura de código e eficácia de testes são coisas diferentes: com 98,64% de cobertura de linhas, a suíte detectava apenas 73,71% das alterações introduzidas. O teste de mutação expôs fraquezas que a cobertura não mostra: asserções que não verificam o conteúdo do erro, ausência de testes nas fronteiras, entradas que "coincidem" com o resultado esperado e ramos de validação que nunca são exercitados.

Cada mutante sobrevivente apontou para uma lacuna específica, o que torna a técnica um guia concreto para melhorar a suíte, e não só uma métrica. Os mutantes equivalentes, por sua vez, revelaram **código redundante** (o if do fatorial e o do produto), outro tipo de achado que a cobertura não aponta. O custo é a execução repetida da suíte para cada mutante, o que aqui foi irrelevante (cerca de 10 segundos para 213 mutantes), mas pode pesar em projetos maiores.